

Marvel - Application de location de films

Laurent Jaures Biveghe

Master 1 TIIL-A - Université de Bretagne Occidentale

- [Marvel - Application de gestion et location de films](#)
 - [Description](#)
 - [Fonctionnalités principales](#)
 - [Structure du projet](#)
 - [Prérequis](#)
 - [Lancer le projet avec Eclipse \(méthode recommandée\)](#)
 - [Compiler et lancer en ligne de commande](#)
 - [Lancer les tests unitaires](#)
 - [Vérification de la qualité du code \(Checkstyle\)](#)
 - [Sauvegarde des données](#)
 - [Documentation](#)
 - [Auteur](#)

Marvel - Application de gestion et location de films

Description

Marvel est une application de bureau en **Java / JavaFX** simulant un service de location de films. Elle propose deux interfaces distinctes :

- une interface **Administration**, pour gérer le catalogue (films, artistes, genres) ;
- une interface **Utilisateur**, pour consulter le catalogue, louer des films et les évaluer.

L'application repose sur une architecture en couches, avec :

- des classes **métier** (`Film`, `Artiste`, `Utilisateur`, `Evaluation`, `Genre`, `InformationPersonnelle` ...) ;
- des **façades** (`FacadeAdministration`, `FacadeUtilisateur`) qui exposent les opérations métier aux contrôleurs ;
- des **interfaces** de contrat (`InterAdministration`, `InterUtilisateur`, `InterSauvegarde`) ;
- des **contrôleurs JavaFX** (`AdministrationContrôleur`, `UtilisateurContrôleur`) associés à des vues FXML ;
- un mécanisme de **sauvegarde / chargement** des données dans un fichier ;
- une suite de **tests unitaires JUnit 5** ;
- une vérification de la qualité du code avec **Checkstyle** (règles Google).

Fonctionnalités principales

Côté administration :

- créer / supprimer un artiste (réalisateur ou acteur) ;
- créer / supprimer un film, lui associer des acteurs et des genres ;
- ouvrir ou fermer la location d'un film ;
- consulter l'ensemble des films et artistes enregistrés.

Côté utilisateur :

- s'inscrire et se connecter à un compte ;
- consulter le catalogue de films (par genre, par acteur, par réalisateur) ;
- consulter les évaluations d'un film et sa note moyenne ;
- louer un film disponible.

Structure du projet

```

├── src/
│   ├── MainLocation.java    # Classe d'essai simple (sans IHM)
│   ├── location/           # Classes métier, façades, exceptions
│   ├── io/                 # Sauvegarde / chargement des données
│   ├── ui/                 # Contrôleurs JavaFX + vues FXML
│   │   └── MainInterface.java # Classe principale (à lancer)
│   ├── tests/              # Tests unitaires JUnit 5
│   └── ressources/         # Images de l'interface
├── doc/                    # Documentation Javadoc générée
├── bin/                   # Classes compilées (Eclipse)
├── Plan de test Marvel.pdf
├── .classpath / .project   # Configuration du projet Eclipse
└── .checkstyle             # Configuration Checkstyle (Google)

```

Prérequis

- **JDK 17** ou version supérieure installé et configuré (`java -version` / `javac -version`).
- **JavaFX SDK** (le projet utilise une bibliothèque utilisateur Eclipse nommée `Java FX`, séparée du JDK à partir de Java 11).
- **JUnit 5**, pour exécuter les tests unitaires (fourni via le conteneur JUnit d'Eclipse, ou à ajouter manuellement en ligne de commande).
- **Eclipse IDE** (recommandé) avec le plugin **Checkstyle** (`eclipse-cs`) si vous souhaitez relancer les vérifications de qualité de code.

Le projet ne contient pas de fichier `pom.xml` ni `build.gradle` : il s'agit d'un projet **Eclipse classique**, géré via les fichiers `.project` et `.classpath`.

Lancer le projet avec Eclipse (méthode recommandée)

1. Ouvrir Eclipse puis **File > Import > Existing Projects into Workspace**, et sélectionner le dossier `V1`.
2. Configurer la bibliothèque utilisateur JavaFX si ce n'est pas déjà fait :
 - **Window > Preferences > Java > Build Path > User Libraries**, créer une bibliothèque nommée `Java FX` pointant vers les fichiers `.jar` du SDK JavaFX téléchargé.
3. Ajouter les arguments VM nécessaires au lancement JavaFX :
 - clic droit sur le projet > **Run As > Run Configurations > Arguments > VM arguments**, puis renseigner :

```
--module-path "<chemin_vers_javafox-sdk>/lib" --add-modules javafx.controls,javafx.fxml
```

4. Exécuter la classe principale :

```
src/ui/MainInterface.java
```

Cette classe lance successivement la fenêtre d'administration puis la fenêtre utilisateur.

Compiler et lancer en ligne de commande

Depuis le dossier `V1`, en remplaçant `<chemin_javafox>` par le chemin vers votre installation du SDK JavaFX :

```

# Compilation
javac --module-path <chemin_javafox>/lib --add-modules javafx.controls,javafx.fxml \
      -d bin $(find src -name "*.java" -not -path "*/tests/*")

# Copier les ressources nécessaires (images et fichiers FXML) dans bin
cp -r src/ressources bin/
cp src/ui/*.fxml bin/ui/

# Exécution
java --module-path <chemin_javafox>/lib --add-modules javafx.controls,javafx.fxml \
     -cp bin ui.MainInterface

```

Classe à exécuter : `ui.MainInterface` C'est la seule classe qui lance réellement l'application graphique (les deux fenêtres Administration et Utilisateur).

`MainLocation.java`, à la racine de `src`, est une classe d'essai minimale sans interface graphique : elle ne sert qu'à vérifier que l'environnement Java fonctionne correctement, elle n'est **pas** le point d'entrée de l'application.

Lancer les tests unitaires

Les tests se trouvent dans le package `tests` et utilisent **JUnit 5**. Sous Eclipse, il suffit de faire un clic droit sur le dossier `src/tests` puis **Run As > JUnit Test**.

En ligne de commande, avec le [JUnit Console Launcher](#) :

```
javac -cp bin:junit-platform-console-standalone.jar -d bin $(find src/tests -name "*.java")
java -jar junit-platform-console-standalone.jar -cp bin --scan-classpath
```

Vérification de la qualité du code (Checkstyle)

Le projet est configuré avec le plugin **Checkstyle for Eclipse**, sur la base des règles **Google Checks** (voir `.checkstyle`). Une fois le plugin installé dans Eclipse, la vérification se lance automatiquement à la compilation, ou manuellement via clic droit sur le projet > **Checkstyle > Check Code with Checkstyle**.

Sauvegarde des données

Les données de l'application (films, artistes, évaluations, utilisateurs) peuvent être sauvegardées puis rechargées dans un fichier via l'interface `InterSauvegarde` (package `io`), grâce aux méthodes `sauvegarderDonnees(String nomFichier)` et `chargerDonnees(String nomFichier)`.

Documentation

Une documentation **Javadoc** du code est disponible dans le dossier `doc/`. Un **plan de test** du projet est également fourni (`Plan de test Marvel.pdf`).

Auteur

Projet réalisé par **Laurent Jaures Biveghe**, dans le cadre d'un projet universitaire (Licence Informatique, Université de Bretagne Occidentale) - durée : 3 mois, 2024.